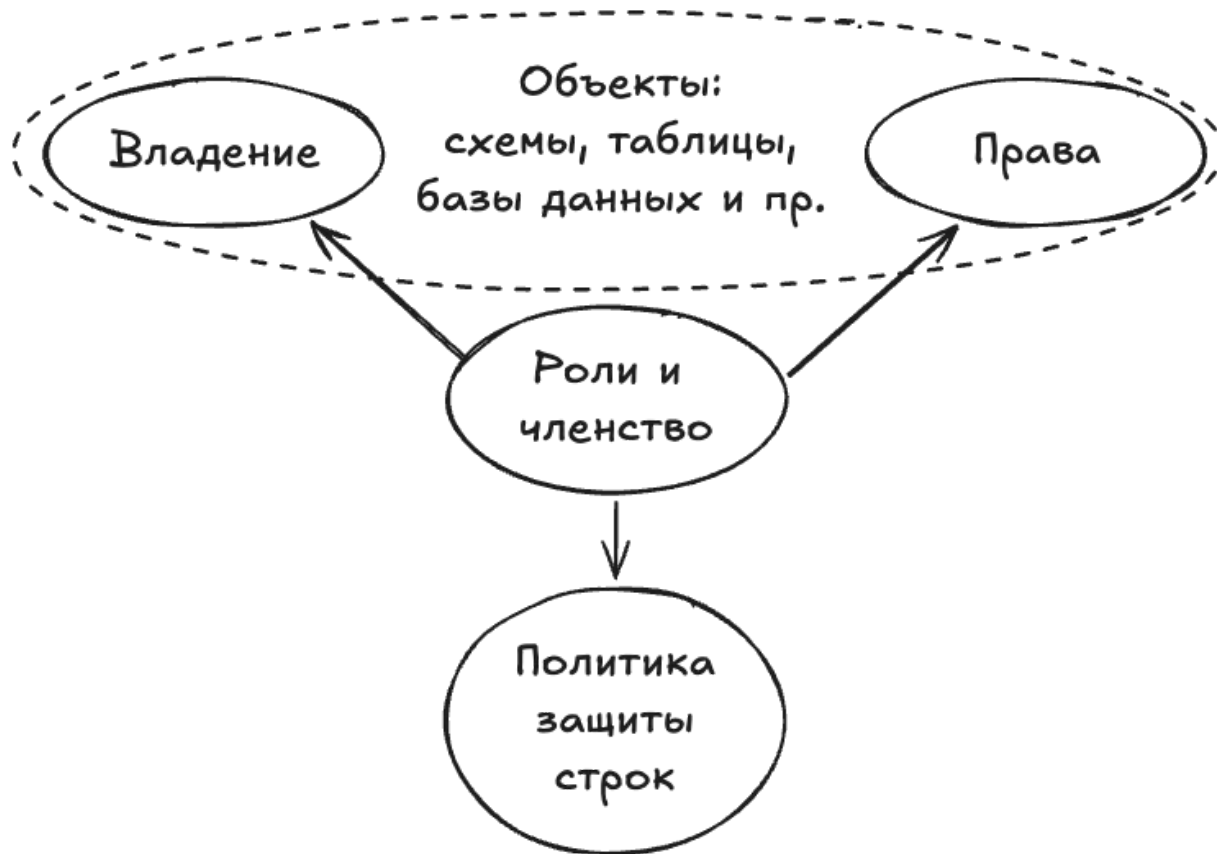


Контроль доступа. Основы DCL.

Лекция 4

Система контроля доступа Postgres



Роли

Роль – это и отдельный пользователь, и группа пользователей

- Может владеть объектами
- Обладает правами (разрешениями) на доступ к объектам
- Может предоставлять членство другим ролям
- Имеет атрибуты, определяющие полномочия

Роль – это глобальный объект кластера!

CREATE ROLE base; – создание роли

ALTER ROLE base **RENAME TO** basic; – переименование роли

\du – в PSQL список ролей

SELECT * FROM pg_roles; - также выведет список ролей

Владение

Когда роль, создает объект, то эта роль автоматически становится *владельцем* объекта. Владелец может делать с объектом все!

Объекты, которыми можно владеть:

- Структура: база данных, схема, табличное пространство
- Данные: таблицы, представления, материализованные представления, последовательности
- Код: функции, процедуры, операторы
- Типы: типы, домены

И многое другое

Передача владения

Передача владения возможна при одном из двух условий:

- Текущая роль – суперпользователь
- Текущая роль – владелец объекта и может выполнить `SET ROLE new_owner;`, где `new_owner` – роль нового владельца

`ALTER TABLE table_name OWNER TO new_owner;` – передача владения таблицей

`ALTER ALTER SCHEMA schema_name OWNER TO new_owner;` – передача владения схемой

`ALTER DATABASE db_name OWNER TO new_owner;` – передача владения базой данных

Проверка владения: через команды `psql \dt` (таблицы), `\dv` (представления), `\dn` (схемы) или через CASE средства

`SELECT datname, pg_get_userbyid(datdba) AS owner FROM pg_database WHERE datname = 'db_name';` – проверка владения БД

Права

SELECT (выборка всех или части столбцов)	CONNECT (подключение к базе)
INSERT (вставка данных)	TEMPORARY (создание временных таблиц)
UPDATE (изменения всех или части столбцов)	EXECUTE (вызов функций)
DELETE (удаление данных)	USAGE (чтение списка объектов схемы, чтобы работать со схемой нужен CREATE)
TRUNCATE (опустошение таблицы)	SET (задание параметров сеанса)
REFERENCES (создание ограничений)	ALTER SYSTEM (конфигурация сервера)
TRIGGER (создани триггеров)	MAINTAIN (VACUUM, ANALYZE, REFRESH MATERIALIZED VIEW, REINDEX, LOCK TABLE)
CREATE (создание схем, установка расширений, создание объектов в схеме)	

Права и объекты

Право	Сокращение	Применимые типы объектов
SELECT	r («read», чтение)	LARGE OBJECT, SEQUENCE, TABLE (и объекты, подобным таблицам), столбец таблицы
INSERT	a («append», добавление)	TABLE, столбец таблицы
UPDATE	w («write», запись)	LARGE OBJECT, SEQUENCE, TABLE, столбец таблицы
DELETE	d	TABLE
TRUNCATE	D	TABLE
REFERENCES	x	TABLE, столбец таблицы
TRIGGER	t	TABLE
CREATE	C	DATABASE, SCHEMA, TABLESPACE
CONNECT	c	DATABASE
TEMPORARY	T	DATABASE
EXECUTE	X	FUNCTION, PROCEDURE
USAGE	U	DOMAIN, FOREIGN DATA WRAPPER, FOREIGN SERVER, LANGUAGE, SCHEMA, SEQUENCE, TYPE
SET	s	PARAMETER
ALTER SYSTEM	A	PARAMETER
MAINTAIN	m	TABLE

Вывод прав объекта

Тип объекта	Все права	Права PUBLIC по умолчанию	Команда psql
DATABASE	CTc	Tc	\l
DOMAIN	U	U	\dD+
FUNCTION или PROCEDURE	X	X	\df+
FOREIGN DATA WRAPPER	U	нет	\dew+
FOREIGN SERVER	U	нет	\des+
LANGUAGE	U	U	\dL+
LARGE OBJECT	rw	нет	\dl+
PARAMETER	sA	нет	\dconfig+
SCHEMA	UC	нет	\dn+
SEQUENCE	rwU	нет	\dp
TABLE (и объекты, подобные таблицам)	arwdDxtm	нет	\dp
Столбец таблицы	arwx	нет	\dp
TABLESPACE	C	нет	\db+
TYPE	U	U	\dT+

Права выводятся в
виде списка

правообладатель=аббревиатура-права[].../правообладатель*

<https://postgrespro.ru/docs/postgrespro/18/ddl-priv>

Если *правообладатель* -
пустое, это PUBLIC

Выдача прав

GRANT *право* ON *object_type* TO *роль*;

```
GRANT USAGE ON SCHEMA library TO lib_user;
```

```
GRANT SELECT, UPDATE, INSERT ON TABLE library.readers TO lib_user;
```

```
GRANT SELECT (card_no, first_name, last_name, reader_category) ON TABLE  
library.readers TO lib_user; – разрешена выборка только отдельных полей
```

```
GRANT ALL ON TABLE library.readers TO lib_user; – разрешаем все действия с  
таблицей
```

```
GRANT SELECT ON ALL SEQUENCES IN SCHEMA library TO lib_user; – разрешаем  
читать из всех последовательностей в схеме library
```

<https://postgrespro.ru/docs/postgrespro/18/sql-grant>

Кто может выдавать права?

- Суперпользователь
- Владелец
- Роль, которая получила права с флагом WITH GRANT OPTION
`GRANT SELECT ON TABLE library.readers TO lib_user WITH GRANT OPTION;`
Теперь lib_user может давать право SELECT другим ролям

Роль PUBLIC

Все роли неявно наследуют псевдо-роль PUBLIC.

Для роли PUBLIC по-умолчанию:

- Таблицы, столбцы, схемы, последовательности, большие объектов, табличные пространства и параметры конфигурации **никаких прав не получают**
- Для баз данных получают права CONNECT и TEMPORARY
- Для функций и процедур получают EXECUTE
- Для типов данных и языков получают USAGE

`GRANT SELECT ON TABLE library.book_loans to PUBLIC;` — мы можем давать права роли PUBLIC

Права по-умолчанию

`ALTER DEFAULT PRIVILEGES...` — позволяет изменить права для всех **будущих** действий

`ALTER DEFAULT PRIVILEGES FOR ROLE lib_manager IN SCHEMA library`

`GRANT SELECT ON TABLES TO lib_user;` — для всех таблиц, которые будут созданы lib_manager в схеме library, выдай права на чтение для роли lib_user

Можно применить также и к роли public:

`ALTER DEFAULT PRIVILEGES IN SCHEMA library GRANT SELECT ON TABLES TO PUBLIC;`

Теперь можно читать из всех таблиц, которые будут созданы в схеме library.

`\ddp` — выведет покажет права по-умолчанию, **только** если они менялись

<https://postgrespro.ru/docs/postgrespro/18/sql-alterdefaultprivileges>

Отзыв прав

REVOKE [GRANT OPTION FOR] *право* ON *object_type* FROM *роль* [CASCADE];

Примеры:

```
REVOKE INSERT ON TABLE library.readers FROM lib_user;
```

Если роль выдавала права с помощью WITH GRANT OPTION, то все такие полученные права являются *зависимыми*.

REVOKE SELECT ON TABLE library.readers FROM lib_user CASCADE; – CASCADE гарантирует, что зависимые права будут также отозваны, иначе – ошибка

REVOKE GRANT OPTION FOR SELECT ON TABLE library.readers FROM lib_user CASCADE;
REVOKE GRANT OPTION FOR отзывает не само право, а лишь возможность передачи

<https://postgrespro.ru/docs/postgrespro/18/sql-revoke>

Удаление ролей

DROP ROLE *роль*; пример: **DROP ROLE** lib_user;

Что будет с объектами, которыми владеет роль?

Можем передать владение конкретного объекта командой

ALTER TABLE library.readers **OWNER TO** new_lib_user; — это неудобно!

REASSIGN OWNED BY lib_user **TO** new_lib_user; — передает владение всеми объектами другой роли. Нужно выполнить в каждой БД.

Удаляем все объекты, которыми владеет роль: **DROP OWNED BY** lib_user;

Также удаляет все права, выданные этой роли.

Важно! Сначала REASSIGN OWNED, затем DROP OWNED. И только после этого DROP ROLE.

<https://postgrespro.ru/docs/postgrespro/18/role-removal>

Членство: наследование прав

Идея! Создадим роль, через которую будем управлять всеми правами, а credenшиалы для доступа будем выдавать через другие роли.

```
CREATE ROLE base WITH LOGIN PASSWORD 'adminpass';  
GRANT USAGE, CREATE on SCHEMA library TO base;  
-- создаем разные таблицы и объекты из-под base  
CREATE ROLE usr1 WITH LOGIN PASSWORD 'usr1pass';  
CREATE ROLE usr2 WITH LOGIN PASSWORD 'usr2pass';  
GRANT base TO usr1;  
GRANT base TO usr2;
```

Роли usr1 и usr2 теперь наследуют все права роли base. Меняя права в base, мы автоматически меняем права ролей usr1 и usr2

Членство без наследования прав

`GRANT base TO usr1 WITH INHERIT FALSE;` – запрещает наследование прав

Тогда зачем?

`GRANT base TO usr1 WITH INHERIT FALSE, SET TRUE;` – еще одна возможность членства – переключаться на другую роль, если установлен флаг `SET`

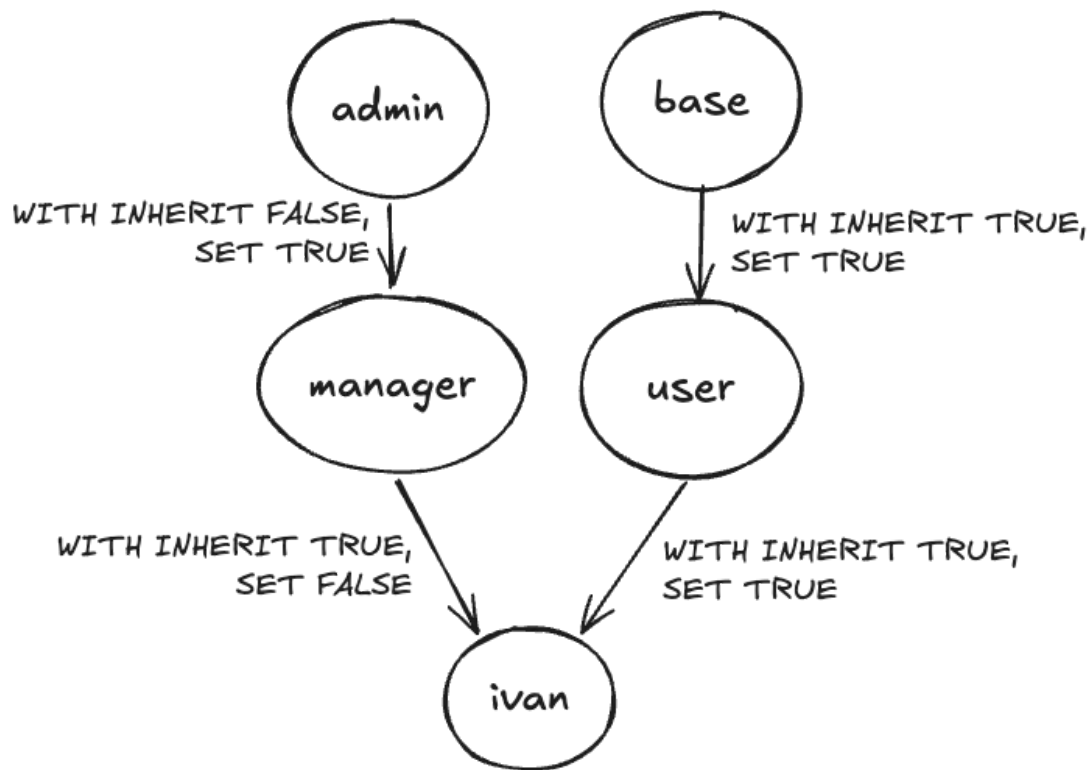
`SET ROLE base;` – теперь `usr1` может стать `base`, чтоб выполнить задачи, требующие админских прав (аналог `sudo`)

`RESET ROLE;` – возврат к роли, из-под которой создавался сеанс

`SELECT current_user;` – покажет, из-под какой роли выполняются команды

Важно: владельцем всех созданных объектов будет `current_user` роль

Членство: пример



ivan наследует права как manager, так и user, и далее по цепочке base

ivan может стать user и base, потому что существует цепочка из SET TRUE

manager не наследует никаких прав admin, но может стать admin

Членство с администрированием

Мы можем считать, что все команды с членством до этого выполнял суперпользователь.

А кто еще может?

`GRANT base TO usr1 WITH ADMIN TRUE;` – теперь `usr1` может управлять членством `base`, т.е. может добавлять/убирать роли-члены в/из `base`

`GRANT base TO usr2;` – теперь `usr1` может выполнить эту команду

`REVOKE base from usr2;` – и эту (убираем членство)

Если нужно поменять опции членства, то `GRANT` выполняется еще раз

`GRANT base TO usr1 WITH ADMIN FALSE;` – теперь `usr1` не сможет управлять членством `base`

Членство: параметры и администрирование

`SELECT roleid::regrole, member::regrole FROM pg_auth_members;` — ПОКАЖЕТ ВСЕ членства (`::regrole` приводит oid к представлению с именем роли)

`SELECT * FROM pg_auth_members;` — ВЫВЕДЕТ ВСЮ информацию о членстве

Параметр	По-умолчанию
<code>WITH INHERIT</code>	Атрибут <code>INHERIT</code> роли-члена
<code>WITH SET</code>	<code>TRUE</code>
<code>WITH ADMIN</code>	<code>FALSE</code>

Атрибуты ролей

Атрибут	Описание
LOGIN	Право подключения: делает из роли пользователя
SUPERUSER	Статус суперпользователя: обходит все проверки прав
CREATEDB	Создание базы данных
CREATEROLE	Создание и удаление роли: только для ролей, у которых членство с ADMIN OPTION
PASSWORD	Пароль
NOINHERIT	Выключение наследования прав, если роль добавляют в члены
BYPASSRLS	Игнорирование политик защиты строк
CONNECTION LIMIT	Максимальное число одновременных соединений (по-умолчанию -1)
REPLICATION	Запуск репликации

Использование атрибутов

```
CREATE ROLE base WITH LOGIN PASSWORD 'adminpass' CREATEDB CREATEROLE NOINHERIT  
BYPASSRLS CONNECTION LIMIT 100; – пример создания роли с атрибутами
```

А как “откатить” изменения для атрибутов? Все просто: приставка NO-:)

```
ALTER ROLE base NOLOGIN PASSWORD NULL NOCREATEDB NOCREATEROLE INHERIT  
NOBYPASSRLS CONNECTION LIMIT -1; – меняем все атрибуты на  
“противоположные”
```

Атрибуты LOGIN, SUPERUSER, CREATEDB и CREATEROLE выглядят, как права, но они не наследуются! Однако ими можно воспользоваться, если переключиться на соответствующую роль с помощью **SET ROLE**

Политики защиты строк (row level security)

`ALTER TABLE readers ENABLE ROW LEVEL SECURITY;` – активируем механизм защиты строк: никакие операции со строками невозможны, пока не определим политики

`CREATE POLICY fresh_records ON library.readers`

`USING (registered_at > '2025-01-01');` – теперь все операции доступны только для читателей, которые зарегистрировались после 2025-01-01

`ALTER POLICY fresh_records ON library.readers USING (registered_at > '2023-01-01');` – обновляем политику

`DROP POLICY fresh_records ON library.readers;` – удаляем политику

Политики принадлежат таблице, имя политики уникально в рамках таблицы

<https://postgrespro.ru/docs/postgrespro/18/ddl-rowsecurity>

Политики защиты строк для операций

Можем определить политики для отдельных операций

```
CREATE POLICY read_all_records ON library.readers FOR SELECT USING (true);
```

— разрешаем читать все записи всем

```
CREATE POLICY correct_passport ON library.readers FOR INSERT TO lib_manager  
WITH CHECK (length(passport_no) = 4);
```

— разрешаем вставку с корректным паспортом и только для роли lib_manager (это пример). Конструкция **WITH CHECK** используется для новых данных, **USING** - для существующих строк

```
CREATE POLICY update_fresh_records ON library.readers FOR UPDATE USING  
(registered_at > '2025-01-01');
```

```
CREATE POLICY delete_fresh_records ON library.readers FOR DELETE USING  
(registered_at > '2025-01-01');
```

- обновление и удаление только свежих записей

Особенности политик защиты строк

- Политики бывают разрешительными (по-умолчанию) и запретительными
- Для разрешительных политик применяется логическое сложение (ИЛИ), для запретительных – умножение (И)
- Владелец, суперпользователь и роль с атрибутом BYPASSRLS обходят проверки политик. Для владельцев можно принудительно включить их.
- Членство работает с RLS также, как и с правами
- Проверки USING и WITH CHECK – SQL выражения, возвращающие bool
- Использовать с осторожностью – избегать сложных SQL выражений, следить за необходимостью индексации полей, которые используются в RLS